

W6 DCGAN

Structure of the session

Any questions left over? Anything to clarify?.....	1
General discussion of what DCGANs and GANs are, and how they fit in with models we have (and have not) seen	1
Types of learning	2
Specific discussion of how DCGANs work.....	2
Examining how this translates into code using the PyTorch DCGAN tutorial	3

Any questions left over? Anything to clarify?

Ask away! + I just remembered that there were some questions I wanted you guys to think about from last week.

General discussion of what DCGANs and GANs are, and how they fit in with models we have (and have not) seen

You may have heard of GANs, “Generative Adversarial Networks.” They’re a type of generative model to create images. DCGANS (“Deep convolutional” GANs) are an improved version of them.

In other words, we’re turning to a new type of model compared to what we’ve seen before. Let’s review the kinds of tasks AI models can do. They can be:

- **Classification models** (e.g. digit classification, such as LeNet)
- **Prediction models** (we haven’t seen any, but linear regression is a prediction model which *can* be solved with machine learning, and predictive policing is an example)
- **Recommendation models** – we haven’t encountered any, but these are the algorithms driving Spotify, content on TikTok etc.
- **Generative models** – we’re about to look at one of them now!

These models have not just use case differences, but they differ in how they “learn”— i.e. what the training process looks like.

But remember: they all rely on machine learning and prediction (how again?).

Types of learning

Supervised: labelled dataset

Unsupervised: no labelled dataset, only patterns are identified (without labels)

(Self-supervised learning: no labelled dataset, but model generates labels)

Reinforcement learning: more used for games, no sense for labelled dataset but algorithm is rewarded for successful task completion, eg. Winning game of chess

On supervised and unsupervised learning, see this helpful guide (with examples):

<https://www.geeksforgeeks.org/machine-learning/supervised-unsupervised-learning/>

On self-supervised learning, see this guide, including code!

<https://www.geeksforgeeks.org/machine-learning/self-supervised-learning-ssl/>

Specific discussion of how DCGANs work

The whole idea behind any GAN (and so also DCGANs) is that there is a generative *adversarial* relationship, meaning a game played between two parts of the model.

Specifically, we've got not one, but two neural networks:

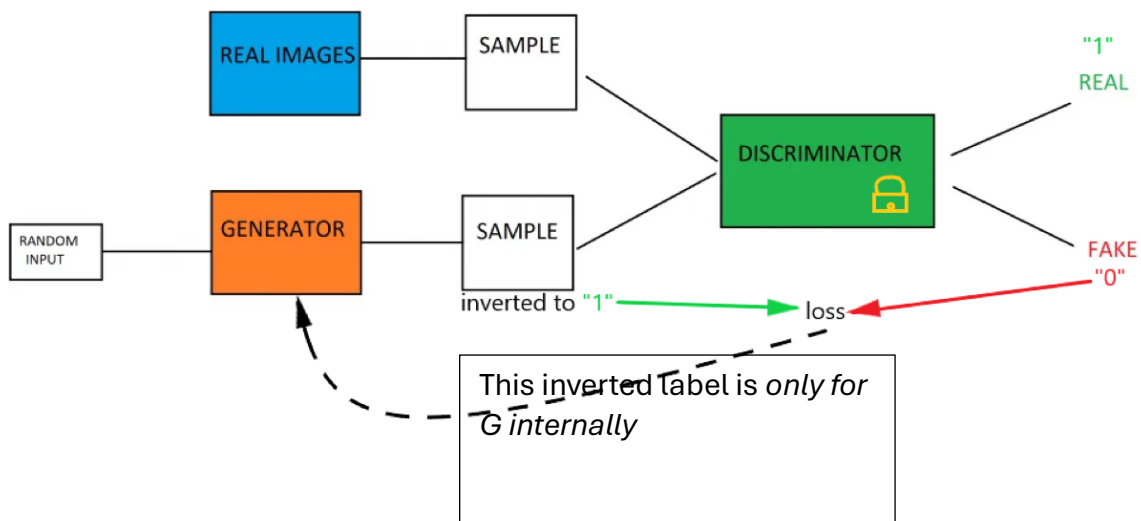
- A **Generator**, which generates images that are supposed to become more "realistic" (=closer to the training data) over time
- A **Discriminator**, which classifies the Generator's outputs as either "real" or generated

The Generator's aim is to fool the Discriminator by producing images that are indistinguishable from the training data. The Discriminator's aim is to get better at telling generated images apart from original images.

So, there are two learning processes:

- The Generator is minimizing the distance of its output from the data. Its loss is big when the Discriminator classifies its outputs as "fake" with a high confidence.
- The Discriminator is more similar to a CNN we've already seen, like LeNet: instead of classifying digits, it's classifying images into one of two categories—"real" or "fake". Its loss is large when its classification is incorrect, that is when it classifies as a generated image as real, or a real image as generated.

Here's a schematic depiction of how that works (sorry, can't remember the source, please don't follow my example ... :/)



Examining how this translates into code using the PyTorch DCGAN tutorial

Here's [the PyTorch DCGAN tutorial](#) (which is not part of the intro YT series); the `dcgan_faces_tutorial.ipynb` file is on Blackboard.

I've also provided you with a dataset (which comes from the Chinese University of Hong Kong, here: <https://mmlab.ie.cuhk.edu.hk/projects/CelebA.html>). Make sure that **you create a new folder inside the folder where your .ipynb file is, and call this new folder "celeba". Extract the data from the .zip file into "celeba".**

Now we're ready to **look at the code!** I can take notes in here for stuff we had questions about, etc.

Note: *don't focus on the specifics of the layers*. That is not the most important (and if you read the original DCGAN paper, you'll see them saying like "We didn't really know what would work best so we just messed around and ended up with these layer sizes.")
→ We don't have time to focus on all the details, so what I want you to focus on is understanding the *general principle* with which these models work.

After training: A note about the ecological implications of these models:

- I only trained this model for two epochs (the tutorial suggested 5), and it took me 177 minutes = 3 hours...
- it nearly drained all of my Mac's battery from a near full charge (and Macs have notoriously good batteries)
- The dataset isn't prohibitively huge (200k images)
- And the results are... okay, I guess?

What do we take away from this?

In order to get good results, we need much, much more data (= all of the internet, basically) and more epochs of training.

→ to be able to actually train that, we need extreme computing power → for this, we need “data centers”

Data centers are immense warehouses housing computer after computer with extremely high-powered processing capability

- Their electricity and water consumption is insane (and their pollution impact is also significant)—though companies are keeping information on this secret
- They’re often in direct competition with humans for those resources (see North London, South Memphis, and Michigan blackouts)
- For specifics on AI’s consumption, see this (funny) page: <https://savethe.ai/>

These resource requirements aren’t specific to DCGANs; they also apply to LLMs, that is, AI chatbots, because those also use massive data (= all of the internet). But whenever image processing is added, this makes the environmental impact shoot up, because images are of course larger data.

There is another implication: data is running out. Because basically all of the internet has already been used up for training, these models can no longer improve. This is why these companies are desperate to find new ways of getting good data. → this is why (among others things) they want your speech/text/image data.

Moreover, much of the internet is now AI-generated—and as you can tell from the Discriminator, no classifier can tell AI-generated images apart from real ones. However, using AI-generated images for model training induces what is called “**model collapse**”: the loss of diversity in model outputs, and even eventual decline in model performance! (See [Shumailov et al. 2024](#))

To understand model collapse intuitively, think back to linear regression. What AI models do, like linear regression, is they approximate a complicated reality by something simpler—expressed as a function. Linear regression approximates the data cloud through a simple straight line. Every time you train a model, its output (such as the regression line) is going to be a simplification compared to the original data. So, when the original data is not real data but something produced by AI, it is already a simplification, which the model we’re training is going to further simplify. And if you keep doing this over and over, at the end you end up with nonsense—like how to imagine something that is simpler than a line?

So, to sum up:

- These AI models are unconscionably harmful to the environment
- Moreover, they actively take away resources (energy, water) from humans who would need them to live

- And when they accomplish the objective for which they do this bad stuff, they basically poison the internet—not just for users of the internet, but also for future models!
- See this argument spelled out more eloquently by leading critical AI scholar Kate Crawford: <https://www.e-flux.com/architecture/intensification/6782975/eating-the-future-the-metabolic-logic-of-ai-slop>

So, what can we do?? Should we just abandon these models?

I'd say no. Three important things:

- No to data centers. Develop, train, and deploy models locally, on your own computers
- Use smaller datasets, go against the race towards the biggest
- Do not use (or use only sparingly) commercially available products like ChatGPT, Claude, Gemini, Copilot, etc., because their use justifies the need for data centers and data extraction (moreover, ChatGPT is solidly pro-Trump).
- Moreover, might there not be pressure that we could mount on these companies to restrict their energy/water consumption etc? → major need for democratic intervention